

Programación 1 — Unidad 3: Estructuras repetitivas

Actividad 2: Máximos, mínimos, continue y break

Índice

- 3.3. break y continue
- 3.5. Patrones frecuentes: acumular, contar y buscar

3.3. break y continue

Dos instrucciones permiten alterar el funcionamiento normal de un bucle. **break** corta el bucle de inmediato, aunque todavía queden vueltas. **continue** salta el resto de la vuelta actual y pasa directamente a la siguiente.

```
for n in [10, 50, 130, 7, 200]:
    if n > 100:
        print(f"Encontrado: {n}")
        break

for n in [4, -2, 8, -5, 3]:
    if n < 0:
        continue
    print(n)
```

Qué hace el código:

- En el primer bucle, se recorre la lista buscando el primer número mayor a 100.
- Al llegar a 130, la condición `n > 100` es verdadera: lo muestra y el `break` corta el bucle de una.
- En el segundo bucle, `if n < 0: continue` saltea los números negativos sin mostrarlos.
- Por eso solo se imprimen los positivos (4, 8 y 3): el `continue` evita llegar al `print` cuando el número es negativo.

3.5. Patrones frecuentes: acumular, contar y buscar

Muchísimos problemas con bucles siguen tres patrones que conviene reconocer porque aparecen todo el tiempo: **acumular** (ir sumando valores), **contar** (cuántos cumplen algo) y **buscar** un extremo (el mayor o el menor).

```
numeros = [4, 9, 2, 7, 5]
```

```
total = 0
for n in numeros:
    total += n
print("Suma:", total)

maximo = numeros[0]
for n in numeros:
    if n > maximo:
        maximo = n
print("Máximo:", maximo)
```

Qué hace el código:

- Para **acumular**: `total` arranca en 0 y, en cada vuelta, `total += n` le suma el número actual. Al final tiene la suma de todos.
- La clave del acumulador es inicializarlo en 0 **antes** del bucle.
- Para **buscar el máximo**: `maximo` arranca con el primer elemento de la lista.
- En cada vuelta, `if n > maximo: maximo = n` reemplaza el máximo si encuentra uno más grande. Al final queda el mayor de todos.